NumPy Arrays & Fast Vectorized Math

Learning Document | Beginner to Intermediate

1. Introduction to NumPy

NumPy (Numerical Python) is a Python library used for numerical computing. It provides a powerful multidimensional array object and tools for performing mathematical operations efficiently. NumPy is widely used in data science, scientific computing, engineering, and machine learning.

2. What Is a NumPy Array?

A NumPy array is a collection of values stored in an organized structure. Unlike a regular Python list, a NumPy array is designed for efficient numerical operations and usually stores elements of the same data type.

Example: Creating an Array

```
import numpy as np

numbers = np.array([10, 20, 30, 40])
print(numbers)
# Output: [10 20 30 40]
```

3. Types of Arrays

One-dimensional array (1D)

```
a = np.array([1, 2, 3, 4])
```

Two-dimensional array (2D)

```
b = np.array([[1, 2], [3, 4]])
```

Three-dimensional array (3D)

```
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

4. Important Array Attributes

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr.ndim)   # Number of dimensions: 2
print(arr.shape)  # Rows and columns: (2, 3)
print(arr.size)   # Total elements: 6
print(arr.dtype)  # Element data type
```

5. Accessing and Slicing Array Elements

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])     # 10
print(arr[-1])    # 50
print(arr[1:4])   # [20 30 40]

matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix[0, 1])  # 2
```

6. Fast Vectorized Math

Vectorization means applying an operation to an entire array without writing an explicit Python loop. NumPy performs these operations using optimized routines, which are often faster and shorter than manually looping through Python lists.

Example: Adding a Number to Every Element

```
arr = np.array([1, 2, 3, 4])
print(arr + 5)
# Output: [6 7 8 9]
```

Common Vectorized Operations

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

print(a + b)   # [11 22 33]
print(a - b)   # [9 18 27]
print(a * b)   # [10 40 90]
print(a / b)   # [10. 10. 10.]
print(a ** 2)  # [100 400 900]
```

7. Useful Mathematical Functions

```
arr = np.array([2, 4, 6, 8])

print(np.sum(arr))   # 20
print(np.mean(arr))  # 5.0
print(np.min(arr))   # 2
print(np.max(arr))   # 8
print(np.std(arr))   # Standard deviation
```

8. Broadcasting

Broadcasting allows NumPy to perform operations between arrays of compatible shapes, including an array and a single number. NumPy automatically expands the smaller shape conceptually without needing to create repeated copies.

```
arr = np.array([10, 20, 30])
print(arr * 2)
# Output: [20 40 60]
```

9. Practical Example: Calculate Student Marks

```
import numpy as np

marks = np.array([80, 75, 90, 85, 70])

total = np.sum(marks)
average = np.mean(marks)
highest = np.max(marks)
lowest = np.min(marks)

print('Total:', total)
print('Average:', average)
print('Highest:', highest)
print('Lowest:', lowest)
```

10. NumPy Arrays vs Python Lists

Feature	NumPy Array	Python List
Main use	Numerical computing	General-purpose collection
Math on whole collection	Direct vectorized operations	Usually requires loops or comprehensions
Data types	Usually one element type	Can contain mixed types
Performance	Often faster for large numerical operations	Can be slower for numerical loops

11. Practice Questions

1. Create a NumPy array containing the numbers 5, 10, 15, 20, and 25.
2. Find the sum and average of an array of five numbers.
3. Multiply every element in an array by 3 using vectorized math.
4. Create a 2D array and display its shape and number of dimensions.
5. Find the maximum and minimum values in an array.

12. Learning Outcomes

After studying this topic, learners should be able to create and inspect NumPy arrays, access array elements, perform vectorized arithmetic, use common mathematical functions, and understand the basics of broadcasting.

Prerequisites: Basic Python variables, lists, loops, and functions.
Suggested practice: Run each code example in Jupyter Notebook or a Python environment with NumPy installed.